

NIT2112 Object-Oriented Programming

Practical Coding Challenge: Smart Warehouse Inventory System

Assessment Type	Open-Book Practical Coding Challenge
Duration	1 Hour
Total Marks	100
Weighting	[Insert %] of Unit Total
AI Copilot Use	REQUIRED (Must be documented)

[!] Important Instructions

1. **Read the entire specification** before beginning to code
 2. **You MUST use an AI Copilot** throughout this challenge - this is assessed
 3. **Document your AI interactions** as you work (copy key prompts/responses)
 4. Save your work frequently
 5. Submit both your Python file AND reflection document
-

[*] Learning Outcomes Assessed

Upon completion, you will have demonstrated the ability to:

- Analyse complex operational requirements and decompose the problem into classes
- Design a robust, maintainable OOP solution
- Implement core logic handling specified rules and interactions accurately
- Apply key OOP concepts: Encapsulation, Inheritance, Polymorphism, Abstraction, and Design Patterns

- Leverage an AI Copilot intelligently as a development partner

[>] Scenario: WareBot Smart Warehouse

You have been contracted by **WareBot Logistics**, a company that operates automated warehouses for e-commerce fulfillment. They need a Python system to manage their inventory, track items, process orders, and handle the different types of storage requirements for various product categories.

The Warehouse Environment

WareBot warehouses contain:

- **Storage Zones:** Different areas optimised for specific product types
- **Inventory Items:** Products with various storage requirements
- **Orders:** Customer requests that must be fulfilled from available stock
- **Alerts:** Notifications when stock levels are critical

Your Task

Design and implement an **Object-Oriented inventory management system** that models this warehouse environment, enforces business rules, and demonstrates your understanding of OOP principles.

[#] Functional Requirements

FR1: Product Hierarchy

Create a product class hierarchy to represent different types of warehouse items:

Product Type	Additional Attributes	Special Rules
Product (Base)	sku , name , quantity , unit_price	Abstract - cannot be instantiated directly
StandardProduct	weight_kg	No special handling required
PerishableProduct	expiry_date , storage_temp	Must check if expired; cannot be shipped if expired
HazardousProduct		

Product Type	Additional Attributes	Special Rules
	hazard_class , handling_instructions	Maximum 50 units per zone; requires certified handler

All products must:

- Have a unique SKU (Stock Keeping Unit)
- Support `get_value()` method returning `quantity × unit_price`
- Implement `is_shippable()` method (polymorphic behaviour)
- Provide `display_info()` method showing product details

FR2: Storage Zones

Implement storage zones with the following capabilities:

Requirement	Description
Capacity	Each zone has a maximum capacity (number of unique products)
Add Product	Add a product to the zone (validate capacity and compatibility)
Remove Product	Remove/reduce product quantity from the zone
Find Product	Search for a product by SKU
Inventory Report	Generate a report of all products in the zone

Zone Types:

- `StandardZone` : Accepts `StandardProduct` only
- `ColdStorageZone` : Accepts `PerishableProduct` only, has `temperature` attribute
- `SecureZone` : Accepts `HazardousProduct` only, has maximum 50 total hazardous units

FR3: Order Processing

Implement order handling with these rules:

Rule	Description
Order Creation	Orders have an <code>order_id</code> , <code>customer_name</code> , and list of order lines
Order Line	Each line specifies a <code>sku</code> and <code>quantity_requested</code>
Fulfillment	Orders can only be fulfilled if all items are shippable and in stock

Rule	Description
Stock Deduction	Successful fulfillment deducts stock from the appropriate zone

Order Status: PENDING → FULFILLED or FAILED

FR4: Low Stock Alerts (Observer Pattern)

Implement a notification system:

- Products should notify observers when quantity falls below a threshold (default: 10)
- WarehouseManager objects can subscribe to products to receive alerts
- Alerts should include the product SKU and current quantity

FR5: Product Factory

Implement a **Factory Pattern** to create products:

```
factory.create_product("standard", sku="SKU001", name="Widget", quantity=100, unit_price=9.99, weight_kg=0.5)
factory.create_product("perishable", sku="SKU002", name="Milk", quantity=50, unit_price=3.50, expiration_date="2023-12-31")
factory.create_product("hazardous", sku="SKU003", name="Bleach", quantity=30, unit_price=12.00, hazard_level="High")
```

FR6: Legacy System Integration (Adapter Pattern)

WareBot has a legacy inventory system that returns product data in an old format:

```
class LegacyInventorySystem:
    def __init__(self):
        self.old_data = {
            "W001": {"item_name": "Hammer", "count": 75, "price_cents": 1599, "mass_grams": 800},
            "W002": {"item_name": "Screwdriver Set", "count": 40, "price_cents": 2499, "mass_grams": 1200}
        }

    def get_legacy_item(self, item_code):
        return self.old_data.get(item_code, None)
```

Create an **Adapter** that converts legacy data into your StandardProduct format:

- price_cents → unit_price (convert to dollars)
- mass_grams → weight_kg (convert to kilograms)
- item_name → name
- count → quantity

[+] Technical Requirements

Your solution **MUST** demonstrate the following OOP concepts:

Concept	Requirement	Marks
Encapsulation	Use private/protected attributes with getters/setters or properties	10
Inheritance	Create a meaningful class hierarchy (min. 3 levels)	10
Polymorphism	Override methods to provide type-specific behaviour	10
Abstraction	Use ABC with at least 2 abstract methods	10
Interfaces	Define at least 1 interface (e.g., <code>Storable</code> , <code>Shippable</code>)	5
Custom Exceptions	Create exception hierarchy for error handling	10
Factory Pattern	Implement product creation via factory	10
Adapter Pattern	Integrate legacy system using adapter	5
Observer Pattern	Implement low-stock notifications	10
Singleton Pattern	Ensure only one <code>WarehouseRegistry</code> exists	5

[!] Custom Exceptions Required

Implement the following exception hierarchy:

```
WarehouseError (base)
├─ InventoryError
│   └─ InsufficientStockError
│       └─ ProductExpiredError
├─ ZoneError
│   └─ ZoneFullError
│       └─ IncompatibleProductError
└─ OrderError
    └─ UnfulfillableOrderError
```

Use these exceptions appropriately throughout your code with meaningful error messages.

[AI] AI Copilot Usage Requirements

You **MUST** use your AI Copilot for:

1. **Design Exploration** - Ask about OOP design options and trade-offs
2. **Code Generation** - Generate boilerplate or complex implementations
3. **Refactoring** - Improve code structure and OOP adherence
4. **Documentation** - Generate docstrings and comments
5. **Debugging** - Identify and fix logical errors

Good AI Copilot Usage Examples [Y]

Prompt Type	Example
Design	"What are the trade-offs between using composition vs inheritance for my Zone classes?"
Implementation	"Help me implement an Observer pattern for low-stock alerts where products notify subscribed managers"
Refactoring	"How can I refactor this code to better follow the Single Responsibility Principle?"
Debugging	"Why might my polymorphic is_shippable() method not be called on PerishableProduct?"

Poor AI Copilot Usage Examples [X]

Issue	Example
Wholesale delegation	"Write the entire warehouse system for me"
No critical thinking	Accepting generated code without understanding or testing it
No documentation	Using AI but not recording how it influenced your solution
Ignoring errors	Pasting AI code that doesn't integrate with your design

[>] Starter Code Template

```
"""
NIT2112 Practical Challenge: WareBot Smart Warehouse System
Student Name: [Your Name]
Student ID: [Your ID]
Date: [Date]

AI Copilot Used: [Yes/No - Name of tool]
"""

from abc import ABC, abstractmethod
from datetime import date

# =====
# CUSTOM EXCEPTIONS
# =====
class WarehouseError(Exception):
    """Base exception for warehouse system errors."""
    pass

# TODO: Implement your exception hierarchy here

# =====
# INTERFACES / ABSTRACT BASE CLASSES
# =====
class Storable(ABC):
    """Interface for items that can be stored in warehouse zones."""

    @abstractmethod
    def get_storage_requirements(self) -> str:
        """Return the storage requirements for this item."""
        pass

class Shippable(ABC):
    """Interface for items that can be shipped."""

    @abstractmethod
    def is_shippable(self) -> bool:
        """Check if the item can be shipped."""
```

```

pass

# =====
# PRODUCT HIERARCHY
# =====
class Product(Storable, Shippable):
    """Abstract base class for all warehouse products."""

    def __init__(self, sku: str, name: str, quantity: int, unit_price: float):
        # TODO: Implement with proper encapsulation
        pass

    @abstractmethod
    def display_info(self) -> None:
        """Display product information."""
        pass

    def get_value(self) -> float:
        """Calculate total value of stock."""
        # TODO: Implement
        pass

# TODO: Implement StandardProduct, PerishableProduct, HazardousProduct

# =====
# OBSERVER PATTERN - Low Stock Alerts
# =====
class StockObserver(ABC):
    """Abstract observer for stock level changes."""

    @abstractmethod
    def update(self, product_sku: str, current_quantity: int) -> None:
        pass

class WarehouseManager(StockObserver):
    """Concrete observer that receives low stock alerts."""

    def __init__(self, name: str):
        # TODO: Implement
        pass

```



```

    def update(self, product_sku: str, current_quantity: int) -> None:
        # TODO: Implement notification handling
        pass

# =====
# STORAGE ZONES
# =====
class Zone(ABC):
    """Abstract base class for storage zones."""

    def __init__(self, zone_id: str, capacity: int):
        # TODO: Implement
        pass

    @abstractmethod
    def add_product(self, product: Product) -> None:
        pass

    @abstractmethod
    def is_compatible(self, product: Product) -> bool:
        pass

# TODO: Implement StandardZone, ColdStorageZone, SecureZone

# =====
# FACTORY PATTERN
# =====
class ProductFactory:
    """Factory for creating different types of products."""

    def create_product(self, product_type: str, **kwargs) -> Product:
        # TODO: Implement factory logic
        pass

# =====
# ADAPTER PATTERN
# =====
class LegacyInventorySystem:
    """Simulates old inventory system with different data format."""

```

```

def __init__(self):
    self.old_data = {
        "W001": {"item_name": "Hammer", "count": 75, "price_cents": 1599, "mass_grams": 800},
        "W002": {"item_name": "Screwdriver Set", "count": 40, "price_cents": 2499, "mass_grams": 1200}
    }

def get_legacy_item(self, item_code: str):
    return self.old_data.get(item_code, None)

class LegacyInventoryAdapter:
    """Adapter to convert legacy inventory data to Product objects."""

    def __init__(self, legacy_system: LegacyInventorySystem):
        # TODO: Implement
        pass

    def get_product(self, item_code: str) -> Product:
        # TODO: Convert legacy format to StandardProduct
        pass

# =====
# SINGLETON PATTERN - Warehouse Registry
# =====
# TODO: Implement WarehouseRegistry as a Singleton

# =====
# ORDER PROCESSING
# =====

class OrderLine:
    """Represents a single line item in an order."""

    def __init__(self, sku: str, quantity_requested: int):
        # TODO: Implement
        pass

class Order:
    """Represents a customer order."""

    def __init__(self, order_id: str, customer_name: str):

```

```

        # TODO: Implement
        pass

    def add_line(self, order_line: OrderLine) -> None:
        # TODO: Implement
        pass

    def fulfill(self, zones: list) -> bool:
        # TODO: Implement order fulfillment logic
        pass

# =====
# MAIN DEMONSTRATION
# =====
if __name__ == "__main__":
    print("=" * 60)
    print("WareBot Smart Warehouse System")
    print("=" * 60)

    # TODO: Demonstrate your implementation
    # 1. Create products using the factory
    # 2. Set up storage zones
    # 3. Add products to zones
    # 4. Set up warehouse manager observers
    # 5. Process sample orders
    # 6. Demonstrate legacy system integration
    # 7. Show exception handling

    pass

```

[#] Sample Expected Output

Your program should demonstrate functionality similar to:

```

=====
WareBot Smart Warehouse System
=====

--- Creating Products via Factory ---
Created: StandardProduct[SKU001] Widget - 100 units @ $9.99

```

```
Created: PerishableProduct[SKU002] Organic Milk - 50 units @ $3.50 (Expires: 2025-02-15)
Created: HazardousProduct[SKU003] Industrial Bleach - 30 units @ $12.00 [Corrosive]

--- Setting Up Storage Zones ---
StandardZone 'ZONE-A' created (capacity: 100)
ColdStorageZone 'ZONE-B' created (capacity: 50, temp: -4°C)
SecureZone 'ZONE-C' created (capacity: 20)

--- Adding Products to Zones ---
SKU001 added to ZONE-A successfully
SKU002 added to ZONE-B successfully
SKU003 added to ZONE-C successfully

--- Subscribing Manager to Low Stock Alerts ---
Manager 'Alice' subscribed to SKU001 alerts

--- Processing Order ORD-001 ---
Order ORD-001 for customer 'John Smith':
  - SKU001: 95 units requested
Checking stock availability...
Fulfilling order...
[!] LOW STOCK ALERT: Manager Alice notified - SKU001 now has 5 units
Order ORD-001 FULFILLED successfully!

--- Legacy System Integration ---
Adapting legacy item W001...
Converted: StandardProduct[W001] Hammer - 75 units @ $15.99

--- Exception Handling Demo ---
Attempting to fulfill order with expired product...
OrderError: Cannot fulfill - SKU002 has expired (expiry: 2024-01-15)

Attempting to add incompatible product to zone...
IncompatibleProductError: StandardProduct cannot be stored in ColdStorageZone

--- Inventory Report ---
ZONE-A Inventory Report:
  SKU001 - Widget: 5 units (Value: $49.95)
  W001 - Hammer: 75 units (Value: $1199.25)
  Total Zone Value: $1249.20
```

[!] Marking Rubric

Criteria	Excellent (100%)	Good (75%)	Satisfactory (50%)	Needs Work (25%)	Marks
OOP Design & Architecture	Elegant design with clear separation of concerns; appropriate use of all required patterns	Good design with minor issues; most patterns implemented correctly	Basic design works but has structural issues	Poor design; patterns misunderstood or missing	/20
Encapsulation	All attributes properly protected; meaningful getters/setters/properties	Most attributes protected correctly	Some encapsulation but inconsistent	Public attributes throughout	/10
Inheritance & Polymorphism	Deep, meaningful hierarchy; polymorphic methods used elegantly	Good hierarchy with working polymorphism	Basic inheritance present	Flat structure or broken inheritance	/15
Abstraction & Interfaces	Well-designed ABCs with clear contracts; interfaces used appropriately	ABCs present with minor design issues	Basic ABC implementation	Missing or non-functional abstractions	/10
Custom Exceptions	Complete hierarchy; exceptions used consistently with helpful messages	Good exception handling with minor gaps	Basic exceptions present	Few or no custom exceptions	/10
Design Patterns	All 4 patterns (Factory, Adapter, Observer, Singleton)	3 patterns correct	2 patterns correct	1 or fewer patterns	/15

Criteria	Excellent (100%)	Good (75%)	Satisfactory (50%)	Needs Work (25%)	Marks
	correctly implemented				
Code Quality	Clean, well-documented, follows Python conventions	Good quality with minor style issues	Functional but messy	Difficult to read or maintain	/5
AI Copilot Usage	Excellent documentation; critical evaluation evident; AI used strategically	Good documentation; AI assisted effectively	Basic documentation; AI helped somewhat	Poor/no documentation; AI not used effectively	/10
Reflection Document	Insightful design rationale; clear AI interaction log with critical analysis	Good rationale and log	Basic reflection	Missing or superficial	/5

Total: /100

[>] Submission Requirements

Submit the following files:

1. `warehouse_system.py` - Your complete Python implementation
2. `NIT2112_Reflection_[YourStudentID].pdf` - Containing:
 - **Design Rationale** (max 500 words): Explain your class design choices, why you structured the hierarchy as you did, and any trade-offs you considered
 - **AI Interaction Log**: Document at least 5 significant AI interactions including:
 - The prompt you used
 - A summary of the AI's response
 - How you used/modified/rejected the suggestion and why

[T] Time Management Suggestion

Phase	Time	Activities
Analysis & Design	10 min	Read spec, sketch class diagram, ask AI for design feedback
Core Implementation	30 min	Implement classes, patterns, exceptions
Testing & Refinement	15 min	Test edge cases, fix bugs, add documentation
Reflection Writing	5 min	Complete design rationale and AI log

[i] Final Tips

1. **Start with the class hierarchy** - Get Product and its subclasses working first
 2. **Test incrementally** - Don't write everything then test; test as you go
 3. **Use AI strategically** - Don't ask it to write everything; ask focused questions
 4. **Handle errors gracefully** - Use your custom exceptions throughout
 5. **Comment your code** - Especially explain non-obvious design decisions
-

Good luck! Remember: This assessment evaluates your ability to design and implement OOP solutions WITH AI assistance, not despite it. Use your AI Copilot as a collaborative partner.

End of Assessment Specification